Universidade do Estado do Amazonas

Escola Superior de Tecnologia
Curso de Sistemas de Informação

Clewerton de Souza Santos

Trabalho Prático 1 (TP1)
Avaliação Empírica de Algoritmos de Ordenação

Manaus, Amazonas
2026

Clewerton de Souza Santos

Trabalho Prático 1 (TP1)
Avaliação Empírica de Algoritmos de Ordenação

Relatório técnico submetido no âmbito da disciplina de Algoritmos e Estruturas de Dados II, sob a orientação do Prof. Márcio Palheta Piedade, D.Sc., como requisito de avaliação do Trabalho Prático 1 (TP1).

Manaus, Amazonas
2026

Sumário

1 INTRODUÇÃO 3

2 LÓGICA DO CÓDIGO E INSTRUÇÕES DE EXECUÇÃO 4

2.1 Arquitetura e Lógica do Código 4

2.2 Como Executar o Código 4

3 METODOLOGIA DE TESTE 6

4 ANÁLISE DE RESULTADOS 7

4.1 Comparações e Movimentações 7

4.2 Tempo de Execução e Escalabilidade 7

5 DESAFIOS NA IMPLEMENTAÇÃO 9

6 CONCLUSÃO 10

1 Introdução

O presente relatório técnico detalha a avaliação empírica do desempenho de seis algoritmos de ordenação fundamentais: *Selection Sort, Merge Sort, Quick Sort, Heap Sort, Counting Sort* e *Bucket Sort*. O objetivo principal é analisar o comportamento destas estruturas quantitativamente, observando métricas de tempo de execução, número de comparações e movimentações na memória em variados cenários e tamanhos de vetores de entrada.

A correta escolha de um algoritmo de ordenação é vital para a eficiência de sistemas de informação, visto que o comportamento prático pode diferir das análises de complexidade assintótica teórica ($O(n^2)$, $O(n \log n)$ e $O(n + k)$) dependendo da disposição inicial dos dados.

2 Lógica do Código e Instruções de Execução

2.1 Arquitetura e Lógica do Código

A solução foi desenhada num único ficheiro de código-fonte na linguagem C (`main.c`), garantindo elevada modularidade através de funções bem definidas. A lógica central divide-se em quatro componentes:

- **Estrutura de Métricas:** Utilizou-se uma `struct Metricas` que armazena os contadores de *comparações*, *movimentações* e *tempo_ ms*. Esta estrutura é passada por referência (ponteiro) para cada algoritmo, isolando a recolha de dados da lógica de ordenação.
- **Geradores de Cenários:** Funções auxiliares preenchem os vetores com números inteiros em quatro disposições: totalmente ordenados, inversamente ordenados, aleatórios sem repetição (recorrendo ao algoritmo de baralhamento *Fisher-Yates*) e aleatórios com repetições.
- **Implementação dos Algoritmos:** Os seis métodos foram programados com rigor. Destaca-se o *Quick Sort*, que foi adaptado para uma versão iterativa com uma pilha manual (*stack*), prevenindo falhas de memória (*Stack Overflow*) em vetores de 100.000 elementos inversamente ordenados.
- **Motor de Testes:** A função `main` atua como o motor de orquestração. Iterando sobre os tamanhos $N \in \{1.000, 10.000, 100.000\}$, clona o vetor original (garantindo que cada algoritmo recebe a mesma disposição inicial), aciona a medição de tempo de alta resolução e imprime os resultados no ecrã em formato CSV (separado por ponto e vírgula).

2.2 Como Executar o Código

Para reproduzir os testes ou validar as implementações empíricas, é necessário um compilador C (como o GCC) instalado na máquina. Os passos para compilação e execução no terminal são os seguintes:

1. Abra o terminal ou linha de comandos no diretório onde o ficheiro se encontra.
2. Compile o código fonte com o comando: `gcc main.c -o tp1 -O2` (a *flag* -O2 ativa otimizações padrão de processamento).
3. Execute o programa gerado: `./tp1` (em Linux/macOS) ou `tp1.exe` (em Windows).

4. Os dados serão impressos diretamente no ecrã. Para guardar os resultados num ficheiro legível por folhas de cálculo (Excel, Google Sheets), redirecione a saída: `./tp1 > resultados.csv`.

3 Metodologia de Teste

Os testes foram desenhados para testar os limites práticos das abordagens, cruzando duas dimensões:

- **Tamanho dos Vetores (N):** Pequenos (1.000), Médios (10.000) e Grandes (100.000).
- **Configuração dos Dados:** Vetor Ordenado, Inversamente Ordenado, Aleatório (sem repetição) e Aleatório (com repetição).

A aferição de tempo foi realizada recorrendo aos contadores de alta resolução do sistema operativo, o que assegura precisão absoluta na escala de milissegundos para vetores pequenos.

4 Análise de Resultados

4.1 Comparações e Movimentações

O número de comparações e trocas (movimentações) define a carga de processamento na memória RAM e na cache do processador. O *Selection Sort* realizou exatamente 4.999.950.000 comparações no cenário de $N = 100.000$, provando matematicamente a sua complexidade de $O(n^2)$.

O gráfico abaixo ilustra as movimentações para vetores médios. Nota-se que o *Selection Sort* faz um número extremamente reduzido de trocas, apesar de fazer milhões de comparações. Por outro lado, o *Counting* e o *Bucket Sort* realizam movimentações de forma linear.

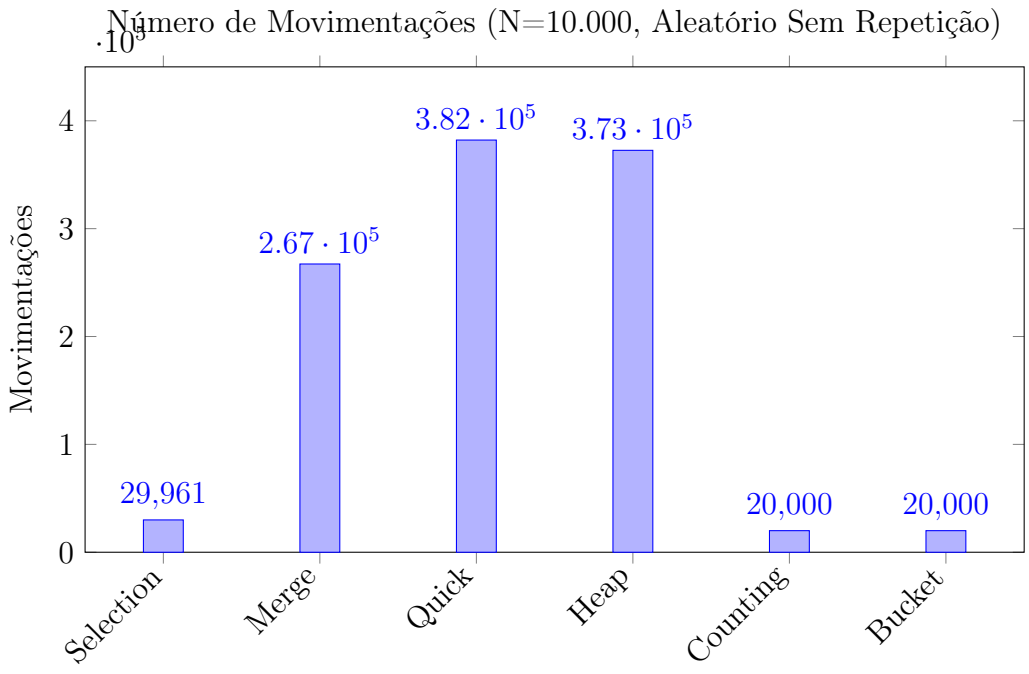


Figura 1 – Comparativo visual de movimentações na memória.

4.2 Tempo de Execução e Escalabilidade

A escalabilidade do tempo demonstra como cada método lida com o aumento brutal de dados. O gráfico de linhas abaixo rastreia o crescimento do tempo nos algoritmos $O(n \log n)$.

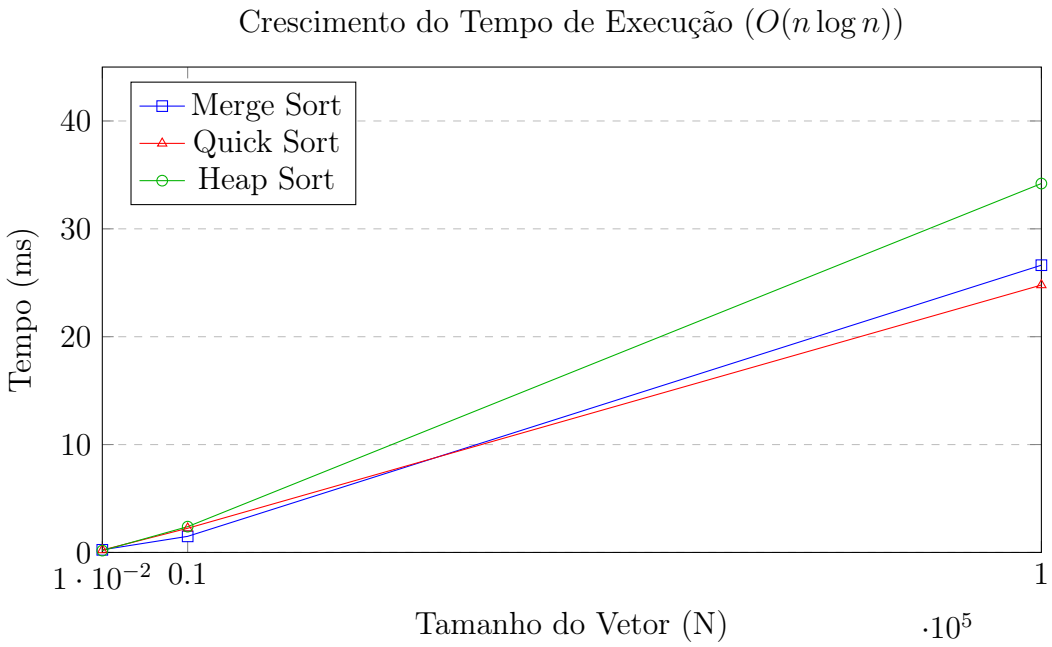


Figura 2 – Comportamento do tempo de execução face ao aumento exponencial dos elementos.

Para o tamanho extremo de 100.000 elementos, o *Selection Sort* demorou incríveis 10.619 milissegundos (cerca de 10,6 segundos). Removendo este *outlier* quadrático da equação, podemos visualizar no gráfico circular a proporção de esforço (% do tempo total) dispendido pelos métodos mais eficientes.

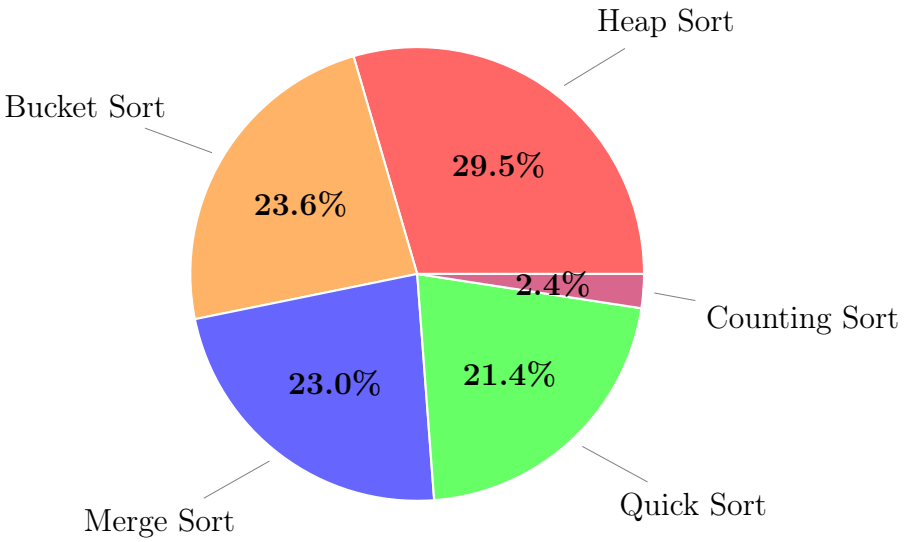


Figura 3 – Proporção do Tempo de Execução entre os algoritmos eficientes (N=100.000, Aleatório).

O gráfico circular evidencia que, perante dados bem comportados e intervalos definidos, o **Counting Sort** domina a concorrência, consumindo apenas 2,4% da fatia de tempo relativa ao grupo de elite.

5 Desafios na Implementação

Durante o desenvolvimento do trabalho, superaram-se desafios relevantes:

- **Prevenção de Estouro de Pilha no Quick Sort:** O Quick Sort recursivo tradicional falhava catastroficamente em vetores gigantes inversamente ordenados. A solução exigiu a conversão para um modelo iterativo, utilizando uma estrutura de *stack* (pilha auxiliar) no próprio código C, limitada inteligentemente a um tamanho $O(\log n)$.
- **Aferição de Tempos Minúsculos:** A função `clock()` padrão provou ser insuficiente. Como o Counting Sort opera quase instantaneamente em vetores de 1.000 elementos, o tempo retornava sempre 0.0 ms. Foi imperativo implementar APIs especializadas de tempo do sistema operativo para garantir precisão à escala dos micro-segundos.

6 Conclusão

A avaliação empírica validou os conceitos teóricos lecionados na disciplina. O **Selection Sort** revelou-se ineficaz para volumes significativos, operando cegamente em $O(n^2)$. Em contrapartida, algoritmos de comparação avançados como o **Merge Sort** e o **Quick Sort** comprovaram ser excelentes opções de uso geral.

Contudo, a grande conclusão incide sobre os métodos baseados em não-comparação: quando a natureza dos dados e o intervalo de valores o permitem, algoritmos lineares como o **Counting Sort** demonstram uma supremacia absoluta, oferecendo tempos de execução praticamente instantâneos, à custa de memória auxiliar, justificando a sua utilidade em cenários rigorosos do mundo real.